03-master-product-spec.md master-product-spec[1]

ProjectEnhancer — Master Product Specification

Document type: Master product definition
Status: v1.0
Product statement: A safe, multi-language AI project enhancement platform that creates an approved Enhanced Version 2 while preserving the user's original project.

1. Core promise

ProjectEnhancer does not silently rewrite a project. It:

```
Connect -> Detect -> Confirm -> Check completeness -> Test baseline
-> Repair approved existing issues -> Validate -> Recommend
-> Preview exact changes -> Get approval -> Create Version 2 -> Validate
-> Deliver diff / pull request
```

2. Primary user journey

1. User signs in with GitHub.
2. Home page opens; repository selection is optional until the user chooses it.
3. User selects one repository from a searchable, paginated list.
4. Detection screen runs a real, visible scan for at least five seconds and reports actual progress.
5. User confirms the detected stack. Unknown technology remains Not detected; no guessing.
6. Completeness check identifies complete, broken, partial, placeholder, and unimplemented areas.
7. User chooses **Complete Project First** or **Enhance Existing Parts Only**.
8. Baseline checks run on the existing project before enhancement.
9. If failures exist, the user receives a separate repair plan.
10. Approved repairs run on an isolated repair branch and are tested again.
11. Only after a stable baseline, or explicit existing-only scope, does enhancement analysis proceed.
12. User reviews paginated recommendations and opens one detailed recommendation at a time.
13. User approves selected changes.
14. The platform creates an isolated Enhanced Version 2 branch.
15. Full validation runs. Failures pause delivery.
16. User receives exact diffs and may create a pull request; merging remains the user's decision.

3. Scope modes

Complete Project First

The platform pauses enhancement and produces a completion plan for missing features. It does not pretend placeholders are working features.

Enhance Existing Parts Only

The platform tests and improves only already-created areas. It may fix bugs, errors, naming, imports, existing UI, folder organization, tests, and documentation. It does not invent missing business features or complete empty

modules.

4. Status vocabulary

Every project area uses one of these statuses:

- **Working** — relevant checks pass.
- **Working with issues** — available and testable, but has known defects.
- **Broken** — implemented behavior fails.
- **Partial** — some implementation exists.
- **Placeholder** — visible shell or dummy content exists.
- **Not implemented** — required code or behavior is absent.
- **Not available** — no suitable check/tool exists.

5. Feature areas

- Repository and GitHub access.
- Multi-language and multi-framework detection.
- Completeness and scope control.
- Baseline build, test, security, and runtime checks.
- Approved repair workflow.
- UI/UX and design review.
- Architecture and folder structure review.
- Security and performance review.
- Testing and documentation recommendations.
- File/component/button/folder impact tracking.
- Before/after UI preview and code diff.
- Enhanced Version 2 branch, manifest, rollback, and pull request.

6. UI product rules

The application uses a single-viewport rule: one primary task per screen. Long forms become wizard steps. Long lists and tables are paginated. Many sections become tabs or routes. Charts are switchable. Mobile shows one primary panel at a time. Content is never clipped with `overflow: hidden`. The marketing landing page may scroll intentionally.

7. Recommendation contract

Each recommendation must show:

```
Title and category
Risk
Current problem
Suggested solution
Affected files
Affected component / button / element / folder
UI, behavior, API, database, and security impact
Files added, modified, moved, or deleted
Expected tests
Approval state
```

8. Safety gates

Enhancement is blocked when the stack is unconfirmed, the repository snapshot is unavailable, required baseline evidence is missing, an approved repair has not validated, or a high-risk change lacks explicit approval. Original branches are never modified directly.

9. Success criteria

The product succeeds when a developer can understand exactly what existed before, what was repaired, what Version 2 changes, why each change was proposed, which tests passed, and how to review or reject the result without risking the original project.

10. Out of scope for the first release

- Automatic completion of unspecified product features.
- Silent database or authentication migrations.
- Universal support without a verified language adapter.
- Claiming tests passed when a test suite was absent or not run.
- Direct production deployment or automatic merge to the original branch.